Aajoo Homes — Response to the Client Document Pack

A response to the seven-document client pack shared on 29 August 2026 — what each document asked for, what we found, what changed, and how the platform behaves now.

Aajoo Homes
Prepared 31 August 2026
Verified against the live platform

One document per client document. For each: what it asked for, what we actually found when we checked, what we changed, and how the system behaves now. Written 2026-08-31 against the live platform.

Everything here was verified against production on the day it was written. Where a claim could not be verified, it says so instead of asserting it.

The pack, and what each file is

Seven files were shared on 2026-08-29 — `Hii.docx` plus six PDFs. They are not seven views of the same list; they are three different kinds of document, and `Hii.docx` says so explicitly. Treating them as one flat backlog was the main risk, so this response keeps them separated.

#	File	Kind	Finding IDs	Answered in
1	`Hii.docx`	Covering note — how to read the rest	—	§1
2	`Aajoo Homes Pricing Architecture.pdf`	**Spec change**	§1–19	§2
3	`AAJOO_HOST_LISTING_SEO_PUBLISH_FIX_SPEC.pdf`	**Spec change**	SEO-01…14	§3
4	`AAJOO HOST LISTING Step 1).pdf`	Flow spec	Nodes 1–N	§4
5	`AAJOO_WEBSITE_FINAL_GUEST_HOST_FINDINGS…pdf`	QA findings	~107 (WEB-P0, G, N, P, C, H)	§5
6	`AAJOO_ADMIN_DASHBOARD_FULL_QA_AUDIT…pdf`	QA findings	37 (ADM, DB, PROD, E2E)	§6
7	`AAJOO_APK_FINDINGS…pdf`	QA findings	36 (FE, BE)	§7

A note on three documents named in `Hii.docx` that were not in this pack. The covering note also describes the *Business Model*, the *POC / Website Design & Flow*, and a *Backend / API / Production Readiness* document. Those were supplied earlier for development and are the "reference basis" the QA documents cite — they were not among the seven files. We used them as reference, not as an action list, which is what the covering note asks for. Where a decision turned on them, it is called out below.

Status in one line: every workstream arising from these documents is complete and deployed. What remains is two client decisions and credential rotation — §9.

§1 — `Hii.docx` (covering note)

What it asked for

That we stop treating the documents as competing requirements and read them as layers: **reference** (what the product is), **QA findings** (what is broken), and **spec changes** (what must be built differently). And it flagged the sequencing risk directly — the pricing document redefines pricing, and everything downstream computes off it.

What we found

The sequencing warning was correct and load-bearing. Negotiation, booking, payment, refunds, host earnings and commission all read from the price. Fixing those first would have meant fixing them twice.

What we did

Pricing was settled first (§2), then negotiation, then everything that consumes a price. Each finding ID is tracked to a numbered part of `REMEDIATION_REPORT_W0-W11_2026-08-31.md`; the ID→location map is PART 13 there.

How it works now

Work is traceable in the shape the covering note asked for: **Finding ID → fix implemented → developer tested → status.** No finding was closed on inspection alone; each has either a test, a live check command, or both.

---

§2 — `Aajoo Homes Pricing Architecture.pdf` (spec change)

The most consequential document in the pack, and not a bug report.

What it asked for

- Three prices per period — **Minimum / Ideal / Maximum**, for **Night, Week and Month**.
- **One pricing engine, two booking experiences.** Real-Time = negotiation; Advance = discount.
- Composite stays: 12 nights = 1 week + 5 nights, computed at each of the three levels.
- The guest sees **only** the Maximum/List price. Minimum and Ideal never leave the server.
- Negotiation: offer **≥ Minimum → auto-accept**; below → host decides.
- **The frontend must never calculate the final price.** UI sends property + dates + mode; the backend returns the breakdown.

What we found

The platform had a single nightly price per listing. There was no week or month tier, no ideal, no composite calculation, and the front end computed its own totals — so the price on the property page and the price the server charged were independently derived and could disagree.

Two figures make the gap concrete: **only 14 of 29,252 listings had a weekend rate, and 6 had a capacity set.** The data to price a stay properly did not exist.

What we changed

- A real pricing engine computing Minimum / Ideal / Maximum across Night / Week / Month, with composite stays.
- `GET /pricing/quote` — server-authoritative. The breakdown the guest sees is the breakdown the server charges.
- A reversible backfill gave **29,239 listings a complete pricing grid**.
- Negotiation rebuilt against the tiers, with every decision logged with a pricing snapshot.
- The 10% deposit shipped as its own payment state — deposit order, balance endpoint, check-in gate, payout held until the balance clears.

How it works now

A guest picks dates. The server computes the three totals, returns only the Maximum as the list price, and the UI renders what it is given. An offer at or above the internal floor is accepted without the host being involved; below it, the host gets Accept / Counter / Decline. The accepted number becomes the booking price and flows into payment — it cannot revert to list.

Two deliberate deviations, both agreed

1. **Negotiation auto-accepts at the Ideal, not the Minimum.** The document specifies the Minimum as the auto-accept threshold. We raised it to the Ideal as a product decision: auto-accepting everything down to the floor gives away the host's entire negotiating range on the first offer. Below-Ideal offers go to the host exactly as specified. **If you want the document's literal rule, it is a one-line change** — say so and we will move it back.
2. **"Advance Booking" as a separate discounted mode does not exist.** The document describes it as a distinct experience with its own discount. On 2026-08-31 you confirmed pre-booking and advance booking are the same thing, and that the 10% deposit already shipped is the whole feature. The dead `ppr_advance_discount` column was retired. What the document called an advance-booking discount is now served by **host and platform offers** (§8).

---

§3 — `AAJOO_HOST_LISTING_SEO_PUBLISH_FIX_SPEC.pdf` (spec change)

What it asked for

Scope change, stated as such: stop building a general CMS SEO system and build a **minimum SEO engine for property listings** — generated when a listing is approved and published, mapped from the host form rather than from a second manual entry surface, with a publication quality gate and no sensitive data in the HTML.

What we found

SEO existed, but it was written to a table nothing served — so pages were being generated and never delivered. Separately, `/property/detail/undefined` was reachable and returned a property-not-found state to crawlers.

What we changed

- SEO is generated **at approval**, from the host form's own fields. One mapping, not two.
- Slug uniqueness enforced at the database level; a slug change automatically creates a 301.
- `robots` rules follow listing status; the sitemap contains only indexable URLs.
- Self-canonical, absolute HTTPS.
- `/property?id=` and `/blog/:id` now 301 to their slug URLs.
- A publication gate blocks anything failing the checks, including a check for sensitive data in the emitted HTML.

How it works now

A listing becomes public → its SEO page exists, is canonical, is in the sitemap, and is reachable at a stable slug. Take the listing down and it leaves the sitemap and stops being indexable. Advanced CMS SEO is deferred, as the document requests.

> **Deploy order matters here:** the frontend must be deployed before the backend, or canonicals point at 404s for the duration of the gap. Recorded in `SESSION_HANDOFF_2026-08-27.md`.

---

§4 — `AAJOO HOST LISTING Step 1).pdf` (flow spec)

What it asked for

A multi-step host listing where **the fields change with the property category**: Host Type → Property Manager details (conditional) → Category → Accommodation Type → Property Name → … with rules like *if PG, hide Entire Property; if Villa, hide Shared Room*, and per-field validation. The stated goal: the host enters information once, and the backend stores structured data that Guest search, property pages, booking, Admin and SEO can all read.

What we found

Two listing forms existed — a legacy one and a partial wizard — and the wizard wrote to modular tables that the guest-facing pages did not read. So a host could complete the new form and the listing would show as having no photos, no amenities, no rules.

What we changed

- The **5-step schema-driven wizard is now the only property form** on web and app. The legacy form is retired on both.
- The schema drives which fields appear per category, server-side.
- The wizard writes **24 modular tables**; a mirror keeps the flat legacy row that renter surfaces read in sync, bridged by a sections API.
- A publish lifecycle with a real state machine: draft → submitted → in review → approved/rejected, with capacity, status and category rules enforced.

How it works now

The host answers once. Category selection changes the remaining questions. Submission puts the listing in review; an admin approves it; approval publishes it and generates its SEO. Admin can no longer approve a draft that was never submitted.

> One consequence worth stating plainly: because the wizard is now the only form, **listings created before it exist as flat rows** and are read through the mirror. That is why the pricing backfill (§2) was necessary.

---

§5 — `AAJOO_WEBSITE_FINAL_GUEST_HOST_FINDINGS…pdf` (~107 findings)

The largest document: six P0s plus Guest auth/search/booking/negotiation/payment/ cancellation and the whole Host side.

What we found

We spot-checked seven of the client's claims against source before starting. All seven were accurate, and one was understated. We then treated the remainder as credible rather than re-litigating each one.

Selected findings and what was actually there:

ID	Claim	What we found
WEB-P0-01	`/property/detail/undefined` reachable	True, and we found the cause: three legacy components still emitted those links, and an `id != null` guard that the string `"undefined"` passes straight through
WEB-P0-04	Booking/payment must be one authoritative transaction	True — the browser could reach a confirmed state on client state alone
WEB-P0-05	Cancellation/refund incomplete	True; worse, cancellation writes ran **outside** their transaction (`{transaction}` passed as the 3rd argument to `Model.update`, which ignores it)
WEB-P0-06	Authorization must be server-side	True
G-19	Availability race	Two guests could book the same dates

ID	Claim	What we found
G-20	Price consistency	The front end computed its own totals — see §2
N-01… N-11	Negotiation must be a real server journey	It was a UI form over a partial backend

What we changed

- **Booking is one authoritative transaction.** Availability → creation → payment → server-side verification → confirmation. Refresh, retry and callback replay are idempotent — they cannot create a second booking or a second charge.
- **Cancellation and refund** are complete: policy shown, OTP required, refund computed server-side, booking state updated, refund status exposed. The transaction bug is fixed in both the guest and host paths.
- **Cancelling now retracts the host payout.** It did not before — live data had ₹19,752 still queued against a cancelled booking.
- **Negotiation** is the full path: offer → decision → final price → booking → payment, with an audit trail. `tbl_negotiation_log` was being written from day one and read by nothing; there is now an admin audit view over it.
- **Authorization** is enforced server-side everywhere: a guest reaches only their own bookings, a host only their own properties, earnings and payouts.
- Search rebuilt on our own place index; dates and guest counts actually filter; guest count validated against host capacity server-side.
- OTP: dev bypass removed, 6 digits, 5-attempt lockout, 10-minute expiry.
- Monthly stays — see §9, this changed after the pack.

How it works now

The guest journey is server-authoritative end to end. Prices, availability, payment state and refunds are all decided on the server; the browser displays results rather than producing them. Tampering with an amount, a booking ID or a URL gets a refusal, not data.

---

§6 — `AAJOO_ADMIN_DASHBOARD_FULL_QA_AUDIT…pdf` (37 findings)

What we found

Worse than reported, in the specific way that matters most.

- **ADM-P0-01 (RBAC on finance routes).** The document said finance routes lack RBAC. In fact **only 3 of 27 routes** in `adminFinance.routes.js` used `requireRole`; the other 24 were authenticated but not authorised — any admin could reach any finance operation.
- **ADM-P0-02 / PROD-01 (test payment credentials).** True, and in *two* places: a Razorpay TEST key was hardcoded as a fallback in `config/db.config.js` and `config/payments.config.js`. On Render, `RAZORPAY_KEY_ID` had never been set — so the "fallback" *was* the live configuration. The consequence is the

expensive kind: checkout opened, the guest "paid", the booking confirmed, an invoice was issued, **and no money was ever collected.**

- **ADM-P0-03 / PROD-02 (CORS).** True. `origin: true` for HTTP and `origin: "*"` for Socket.IO, each with a "consider restricting in production" comment.
- **ADM-P0-04 / DB-01 (RBAC storage).** True — `tbl_admins` had only `admin_isAdmin` and `admin_isActive`. No role column existed to enforce against.
- **ADM-P1-01 (rate limiter).** True. It read `req.admin?.id` while tokens carry `adminId`, so per-admin buckets silently degraded to per-IP.
- **PROD-05.** `/db-test` was publicly exposed.
- **"Disputes".** There is no dispute feature and never was — what existed was a KYC compliance queue wearing the word. Said plainly rather than reported as fixed.

What we changed

- RBAC applied across all 27 finance routes, with a real role column behind it.
- Payment credentials moved to environment-first; **the deploy now fails closed** — a production deploy holding a test key refuses to take payments rather than completing without collecting. `/health` reports whether the deploy can actually take money.
- CORS restricted; `/db-test` removed from public reach; rate limiter reads the correct claim.
- A durable **mutation ledger** (`tbl_admin_audit`): finance mutations require a reason and are recorded with the admin who made them.
- Dashboard KPIs, finance reconciliation, negotiation control plane and the support queue all reworked against real data.
- `po_on_hold` was write-only — admin holds set it and every reader ignored it, counting held money as payable. All three readers now honour it.

How it works now

Admin is role-gated per route. Finance mutations are attributable and reversible. The deploy will not silently take fake payments. The negotiation control plane identifies guest and host by **who owns the property**, not who moved last.

---

§7 — `AAJOO_APK_FINDINGS…pdf` (36 findings)

What it asked for

That the production APK use production configuration and carry no server secrets or development residue, and that the backend contract questions (FE-01…18, BE-01…18) be resolved before the next build.

What we found

- **P0-01.** There was no prod/dev split at all — only the appearance of one. Three constants held the same string, and `baseUrl` was wired to the one named `_dev`, so a release build shipped whatever was last edited into the dev line.
- **P0-02.** Razorpay test key in the build, same root cause as ADM-P0-02.
- **FE-11…18.** Token storage, TLS validation, no-op buttons and hardcoded demo content — all confirmed. The app carried twelve hardcoded `"4.5"` ratings, a literal `· 164` review count, and "Free Cancellation" on every card.
- **Two live regressions our own earlier work had caused**, found here and worth naming: after cancellation OTP was added server-side, **the app could not cancel a booking at all**; and after the deposit option shipped, **app pre-booking recorded stays at a tenth of their price.**

What we changed

- One base URL with a build-time override (`--dart-define=API_BASE_URL=…`), so a release build cannot inherit a developer's edit.
- Payment key handled the same way; secure token storage; TLS checked and surfaced so a build can be verified rather than assumed.
- Stock imagery and fabricated ratings removed; ratings come from `tbl_reviews` and unrated listings render **"New"**, not `0.0`.
- A "Skip for now (dev)" button on signup step 3 that submitted `doc_number: 000000000000` as an Aadhaar number was removed — real accounts were being created with fabricated government-ID data.
- Every signup was sending `user_ref: "0000"`, a hardcoded placeholder that overwrote whatever the referral screen collected. No mobile signup had ever been attributed to a referrer.
- Both regressions above fixed and re-verified end to end.

How it works now

The current build is **1.0.0 (build 7)**, signed with the release key, installed and smoke-tested against production: login, guest dashboard, host portal, bookings, payouts, reviews. No crashes or exceptions in logcat.

> **The APK is a distribution question, not a readiness one.** Pushing source does not ship an app. Every app fix listed here reaches a real host or guest only when a build is actually distributed.

---

§8 — Work that these documents did not ask for

Three pieces shipped in the same period, from conversation and decisions rather than the pack. They are listed so the client's document set and the platform match up.

Work	Why
Pay-at-property settlement	A third payment option, plus the machinery to bill the host their commission + GST on cash bookings and recover it from the next payout. One active pay-at-property booking per renter.
Property offers (discounts)	Your "discounts" ask turned out to be three things. Negotiation and platform coupons already existed; the new one is a host/admin-set **displayed** discount with a strikethrough, a time window, a slot cap and a buffer. A host offer wins on its own listing over any coupon, and a discounted listing is not negotiable.
Web ⇄ mobile parity pass	Not on anyone's list. An endpoint audit across all 385 backend routes and both clients, then a screen-by-screen walk with both accounts signed in on both platforms at once. Fourteen defects — the worst being Host Payouts reading a **different ledger** on each platform: the website showed ₹30,333.16 pending across ten payouts while the app showed ₹0 and "No Payout History". Full account in PART 19 of the remediation report and `WEB_MOBILE_PARITY.md`.

---

§9 — What is still open

Needs a decision from you

1. **Live Razorpay credentials.** Production fails closed without them — this is deliberate, and it is the last thing between the platform and taking real money.
2. **Is the APK in this release candidate?** Build 7 exists and is tested; the question is distribution.

Needs an action on your side

3. **Credential rotation.** Deferred at your instruction until testing is complete. Every secret in `config/db.config.js` is in git history and must be rotated: database password, Cloudinary, Gmail app password, `JWT_SECRET`, Razorpay, DIDIT, Brevo, Firebase.
4. **Four RazorpayX environment variables** (`RAZORPAYX_KEY_ID`, `_KEY_SECRET`, `_ACCOUNT_NUMBER`, `PAYOUT_ENCRYPTION_KEY`). Payouts are built and blocked on these; they also block the one untested path in the cash feature.
5. **At go-live**, remove `ALLOW_TEST_PAYMENTS` and swap in `rzp_live_` keys.

Deferred by decision, with reasons

- **Money columns are `DOUBLE(10,2)`, not `DECIMAL`.** Wrong in principle. We measured rather than assumed: there is no drift today, every paid booking reconciles, nothing is over-credited. Migrating 14 columns on a live database mid-testing carries more risk than it removes.
- **Customer disputes** — no feature exists. Building it properly is its own piece of work, not a bug fix.
- **The property page still renders its own price breakdown** rather than consuming `/pricing/quote`. Checkout is already server-authoritative; switching the display needs a fallback so a quote outage cannot blank a price.
- **The app derives some host figures** from unpaginated endpoints instead of the summary endpoints. Totals agree — that was checked — but it pulls every booking to show one number.

Resolved after this document was first written

- **The encoding question — checked, and it found something else.** The replacement character turned out to be **ours**: the stored bytes are `EF BF BD`, U+FFFD itself, in columns that are `utf8mb4` and would have taken an em-dash. All twelve affected rows read *"automated verification … not a real stay"* — our own test text, mangled by the shell that wrote it before it reached the database.

  The check turned up a real fault that was not the one being looked for. `tbl_messages.message` **— guest ⇄ host chat — was** `utf8mb3`**,** three-byte UTF-8. Devanagari fits in three bytes, so Hindi always worked and this survived every round of testing; an emoji is four, and with `STRICT_TRANS_TABLES` MySQL refuses the row rather than truncating it. A guest sending *"Thanks! See you soon 😊"* had the write rejected — and the socket handler caught the error and emitted `messageSent`, **the success event**, so they were told it had been delivered.

  Both halves are fixed: the table is converted (migration applied to the live database; emoji, Hindi and mixed text now round-trip byte-exact) and a failed send emits a distinct `messageFailed` carrying the text back. `scripts/checkTextEncoding.js` re-checks the live schema on demand.

---

§10 — Two things worth knowing about how this was verified

A caught error can make a broken feature look finished. Adding a cover photo to a payload appeared to work — the stock placeholder was gone and a neutral tile appeared. It was completely broken: a helper resolved to `undefined` in the deployed process, threw into a `catch` that logs and continues, and the endpoint answered **200 with the new field entirely absent**. The page rendered its fallback and looked right. It was caught only by reading the API response instead of the page.

Where two surfaces disagree about a number, at least one is lying, and both may be. Most of the parity defects were of this shape — the same host shown 18 cancellations on one page and 21 on another, "Total Spent" reading ₹34,845, ₹28,125 and ₹26,760 on three different screens. None was caught by a unit test. Each was found by opening both and looking.

Both lessons are recorded so they do not have to be relearned.

Prepared 2026-08-31. Companion documents: `REMEDIATION_REPORT_W0-W11_2026-08-31.md` *(what was done, with verification commands),* `RELEASE_CANDIDATE_TASKLIST_2026-08-29.md` *(workstream planning),* `WEB_MOBILE_PARITY.md` *(web ⇄ app ledger).*